תרגיל בית 2: קליע עובר מעל קיר

תכנות מדעי לפיסיקאים, שבוע 2

הגשה: מגישים את המקטע בשאלת ה־CodeRunner של שבוע 2 (ליבה) בעמוד ה-Moodle של הקורס.

מטלת החובה: האם קליע עובר מעל קיר?

נשגר מגובה הקרקע במהירות v_0, בזווית θ ובגודל כבידה $g > 0$, כאשר $+x$ לכיוון הקיר ו-$+y$ כלפי מעלה. במודל של כבידה אחידה וללא גרר,

$$v_x = v_0\cos\theta, \quad t_w = x_w/v_x, \quad y_w = v_0\sin\theta\, t_w - \tfrac{1}{2}g t_w^2, \quad v_{y,w} = v_0\sin\theta - g t_w.$$

נדרוש $v_x > 0$ לפני החלוקה. נשווה את y_w לגובה הקיר, ונשתמש ב-$v_{y,w}$ כדי לסווג עלייה או ירידה. שוויון בגובה נחשב מעבר; מהירות אנכית אפס שייכת לקטגוריית הירידה שפורסמה. הגובה המחושב בתנועה חופשית עשוי להיות שלילי: עבור גובה קיר שאינו שלילי פירוש הדבר אי-מעבר, ולא תנועה הנמשכת פיסיקלית דרך הקרקע.

תיבת פיסיקה: קליע בקיר

קליע המשוגר מן הראשית נמשך כלפי מטה על ידי הכבידה, ולכן מהירותו האופקית נשארת קבועה $v_x = v_0\cos\theta$ ללא גרר, והמהירות האנכית קטנה. בכבידה אחידה $v_y(t) = v_0\sin\theta - gt$, והגובה במרחק אופקי x_w הוא $y_w = v_0\sin\theta\, t_w - \tfrac{1}{2}g t_w^2$ כאשר $t_w = x_w/v_x$. כאן v_0 היא מהירות השיגור ב-$\mathrm{m/s}$, θ הזווית מעל האופק במעלות, $g > 0$ גודל הכבידה ב-$\mathrm{m/s^2}$, x_w ו-y_w אורכים במטרים, t_w זמן בשניות, ו-$v_{y,w}$ מהירות אנכית ב-$\mathrm{m/s}$. אנו בוחרים $+x$ לכיוון הקיר ו-$+y$ כלפי מעלה, משגרים מגובה הקרקע ודורשים $v_x > 0$. המודל מניח מסה נקודתית, קרקע ישרה, כבידה אחידה וללא התנגדות אוויר. עבור $v_0 = 20\ \mathrm{m/s}$, $\theta = 45^\circ$ ו-$x_w = 10\ \mathrm{m}$ הוא נותן $y_w = 7.5475\ \mathrm{m}$. גובה מחושב שלילי פירושו שהפרבולה של התנועה החופשית חצתה את הקרקע לפני הקיר; זו אינה תנועה הנמשכת מתחת לקרקע. גרר אוויר, סחרור, רוח וכל התנגשות עם הקיר נמצאים מחוץ למודל.

תחילה נכתוב תחזית למקרה רגיל ולמקרה גבול. לאחר מכן נממש את המקטע הסקלרי, נאמת את התחום המתועד שלו, ונשווה את התוויות והערך הבוליאני בדיוק. גבהים נבדוק בקירוב לפי הסבולת שפורסמה. לבסוף נסביר כיצד החלטת המעבר והחלטת המצב האנכי נבדלות זו מזו: קליע יכול לעבור מעל הקיר בעודו יורד. דף המטלה הוא המקום שבו מפורטים שמות הקלט המדויקים, הבדיקות, מוסכמת הפונקציה העוטפת והוראות ההגשה; אין צורך בפונקציות או בלולאות שמגדיר הסטודנט.

המשימה

קליע מתחיל בראשית במהירות speed_m_s ובזווית שיגור angle_deg. במודל האנליטי ללא גרר, נחשב את גובהו בקיר אנכי ונחליט אם הוא עולה או יורד שם ואם הוא עובר את הקיר. נבחר $+x$ אופקי ו-$+y$ כלפי מעלה, $g > 0$, ונמיר זוויות ממעלות לרדיאנים. בקיר, $t = x/(v\cos\theta)$, $y = v\sin\theta\, t - gt^2/2$, ו-$v_y = v\sin\theta - gt$. שוויון עם גובה הקיר נחשב מעבר; $v_y = 0$ מסווג "falling" לפי החוזה. אנו מניחים כבידה אחידה וללא גרר; המודל אינו כולל פגיעה בקיר או בקרקע.

ההגשה הצפויה

מגישים מקטע בלבד (פונקציות נלמדות בשבוע 5). הפונקציה העוטפת מחזירה שלושה ערכים עם הסוגים והערכים האפשריים הבאים:

- `height_at_wall` (ערך Real סופי, בדרך כלל Float64): הגובה המחושב במטרים. הוא עשוי להיות שלילי כאשר הקליע נוחת לפני הקיר.
- `vertical_state` (String): בדיוק "rising" כאשר המהירות האנכית חיובית, או "falling" אחרת. מהירות אנכית אפס היא "falling".

- `clears` (Bool): `true` כאשר `height_at_wall >= wall_height_m`, ו-`false` אחרת. שוויון בגובה הקיר נחשב `true`.

נאמת קלטים ממשיים סופיים שאינם בוליאניים: זווית השיגור בתחום $[0^\circ, 90^\circ)$, המהירות, מרחק הקיר וגובה הקיר אינם שליליים, הכבידה חיובית, והמהירות האופקית חייבת להיות חיובית. (ב-Julia, `Bool` הוא `Real`, ולכן נדחה אותו במפורש.) קלט לא תקין חייב להעלות `throw(ArgumentError(...))`, ולא להדפיס אזהרה. נציב את כל שלושת הערכים; לא נדפיס דבר.

```
# supplied: speed_m_s, angle_deg, wall_x_m, wall_height_m, gravity_m_s2
# assign: height_at_wall, vertical_state, clears
```

כיצד למלא את התשובה

תיבת התשובה ב־CodeRunner כבר מכילה את שמות הקלט ואת שלושת שמות הפלט. השאירו את המבנה ואת השורות המסופקות ללא שינוי. החליפו רק את השורה או ההערה שמבקשת מכם לכתוב את המקטע. אין להגדיר בעצמכם את `wall_clearance`, אין להוסיף `return` ואין להדפיס דבר. הפלטפורמה קוראת את הפונקציה העוטפת ואת שלושת המשתנים שתציבו.

סדר חישוב מומלץ

בתרגיל הראשון ב־CodeRunner כדאי לעבוד בסדר הזה:

1. **בדקו כל קלט.** השתמשו בשרשרת `elseif`/`if` או בפקודות `if` נפרדות. כל ערך חייב להיות `Real` סופי שאינו `Bool`. התחומים המותרים:
   - $\texttt{speed_m_s} \geq 0$
   - $0 \leq \texttt{angle_deg} < 90$
   - $\texttt{wall_x_m} \geq 0$
   - $\texttt{wall_height_m} \geq 0$
   - $\texttt{gravity_m_s2} > 0$

   אם בדיקה נכשלת, הפעילו `throw(ArgumentError("..."))`. הנוסח שבחרתם אינו נבדק; סוג השגיאה כן נבדק.
2. **המירו את הזווית לפני השימוש ב־`sin` או `cos`.** השתמשו ב־`sind` ו־`cosd`, או חשבו `theta = deg2rad(angle_deg)` והשתמשו ב־`theta`.
3. **חשבו את המהירות האופקית ובדקו אותה לפני החלוקה.** הציבו `vx = speed_m_s * cos(theta)`. אם `vx` אינו סופי וחיובי, הפעילו `ArgumentError`; אחרת $\texttt{vx} > 0$ מובטח לחלוקה. חשבו

$$t = \frac{\texttt{wall_x_m}}{\texttt{vx}}, \qquad \texttt{height_at_wall} = \texttt{speed_m_s}\sin\theta\, t - \tfrac{1}{2}\texttt{gravity_m_s2}\, t^2.$$

4. **חשבו את המהירות האנכית וסווגו אותה.** הציבו `vy = speed_m_s * sin(theta) - gravity_m_s2 * t`. הציבו `vertical_state = "rising"` כאשר `vy > 0`; אחרת הציבו "falling". המקרה $\texttt{vy} == 0$ שייך ל־"falling".
5. **הציבו את הערך הבוליאני.** הציבו `clears = height_at_wall >= wall_height_m`. השתמשו ב־`>=` ולא ב־`>`, מפני שפגיעה בדיוק בקצה העליון נחשבת מעבר.
6. **עצרו.** אין להדפיס, להחזיר טאפל או לשנות את המשתנים המסופקים. הפלטפורמה קוראת את `height_at_wall`, `vertical_state` ו־`clears` לאחר הרצת המקטע.

לפני ההגשה: בדקו את המקרים האלה

בכל המקרים `gravity_m_s2 = 9.81`. אלה בדיקות ידניות שימושיות; שלושת הראשונים קרובים לבדיקות הפומביות. הגובה הוא הערך המוחזר, לא שורת פלט.

speed	angle	wall x	wall h	`height_at_wall`	`vertical_state`	`clears`
20	45	10	7	7.5475	`"rising"`	`true`
20	45	30	8	7.9275	`"falling"`	`false`
20	0	0	0	0.0	`"falling"`	`true`
20	45	50	2	-11.3125	`"falling"`	`false`

המקרה השני מראה שקליע יכול לרדת ועדיין לפגוע בקיר. השלישי הוא מקרה הגבול המדויק: הגובה המחושב שווה לגובה הקיר, ולכן clears הוא true, והמצב האנכי הוא "falling" מפני ש־$vy == 0$. במקרה הרביעי הגובה המחושב שלילי; החוזה מתיר זאת, והמשמעות היא שהקליע היה נוחת לפני הקיר.

בדקו גם ערכים שנדחים. לפחות אחד מהמצבים הבאים חייב להעלות ArgumentError:

- מהירות שלילית, או מהירות אפס כאשר המהירות האופקית אינה חיובית;
- זווית שלילית, או זווית של $90°$ ומעלה;
- מרחק קיר שלילי או גובה קיר שלילי;
- כבידה אפס או שלילית;
- NaN או Inf באחד הקלטים;
- ערך בוליאני כגון true במקום מספר.

טעויות נפוצות בתרגיל ראשון

- שימוש ב־sin(angle_deg) או cos(angle_deg) כאשר הזווית במעלות. השתמשו ב־cosd/sind, או המירו עם deg2rad.
- חלוקה ב־vx לפני שבדקתם שהוא חיובי.
- בדיקת value isa Real בלי לזכור ש־Bool הוא גם Real; יש לדחות Bool במפורש.
- כתיבת `clears = height_at_wall > wall_height_m`, שמאבדת את מקרה הקצה המדויק.
- עיגול הגובה לפני ההצבה או ההשוואה. החזירו את הערך המחושב המלא; הבדיקות מתירות הפרשי נקודה צפה רגילים.
- הדפסת תוצאה או הוספת `return height_at_wall, vertical_state, clears`. הפונקציה העוטפת מחזירה את הערכים, והפלט אינו חלק מהחוזה.
- שימוש ברישיות שגויה במחרוזת, כגון "Rising", או הצבת המחרוזת "true" במקום הערך הבוליאני true.

כאשר בדיקת CodeRunner נכשלת, קראו את המקרה שנכשל ואת הערכים הצפוי והמתקבל. השוו את סדר הקלט, יחידת הזווית, סימן הכבידה, בדיקת vx ומוסכמת הגבול. תקנו בעיה אחת והריצו שוב את Check; בדיקות הסגל משתמשות באותו חוזה עם ערכים רגילים וגבוליים אחרים.

כיצד נבדק התרגיל?

הבדיקות הפומביות קוראות לפונקציה העוטפת עם מקרה עולה רגיל, מקרה יורד, מקרה קצה מדויק וכמה קלטים לא תקינים. גבהים מושווים בסבולת 1e-12 מטר; מחרוזת המצב והערך הבוליאני מושווים בדיוק. בדיקות הסגל משתמשות באותו חוזה מפורסם ומוסיפות ערכים רגילים, גבוליים ופסולים אחרים. הן אינן מוסיפות שם קלט חדש, פלט חדש או מוסכמת החזרה אחרת. גוף הבדיקות הפומביות המלא מופיע בסעיף המתקדם (רשות).

מתקדם (רשות)

הפונקציה העוטפת ובדיקות ה-@test שייכות למערכת, ואיננו כותבים אותן השבוע. פונקציה עוטפת היא פונקציה מסופקת שקוראת לקוד המוגש ומחזירה את תוצאותיו; בדיקה שנכתבת עם @test מצהירה על טענה שחייבת להתקיים, ואילו @test_throws מצהיר שקלט פסול חייב להידחות. קריאה בהן מראה לנו כיצד העבודה שלנו נבדקת באותו אופן בכל פעם. גוף הבדיקות הגלויות המלא הוא:

```
@testset "week02 core public" begin
    h, state, clears = wall_clearance(20.0, 45.0, 10.0, 7.0)
    @test isapprox(h, 7.5475; atol=1e-12); @test state == "rising";
@test clears
    h, state, clears = wall_clearance(20.0, 45.0, 30.0, 8.0)
    @test state == "falling"; @test !clears
    @test wall_clearance(10.0, 0.0, 0.0, 0.0) == (0.0, "falling", true)
    @test_throws ArgumentError wall_clearance(-1.0, 30.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(10.0, 30.0, 2.0, -1.0)
    @test_throws ArgumentError wall_clearance(10.0, 90.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(true, 30.0, 2.0, 1.0)
end
```